

# Plan Maître — Réseau Vivant ToM (V2 — 2026-04-01)

CE DOCUMENT EST LE FIL CONDUCTEUR. Il ne peut être ni contourné, ni réinterprété, ni "adapté" par un agent, un LLM, ou un copilote. Chaque étape se suit dans l'ordre. Chaque critère de sortie est un verrou. Pas de raccourci.

## Pourquoi ce plan

On a beaucoup avancé sur le code. Mais rien n'est commité, on ne voit pas ce qui se passe dans le réseau, et on n'a jamais fait tourner plus de 2 machines ensemble.

Ce plan corrige ça : sauver le travail, voir tout, faire vivre le réseau, le casser, et vérifier qu'il se répare.

**Validé par GPT 5.4** : aligné avec le plan maître existant ( 2026-03-31-plan-maitre-reseau-vivant-chaos.md ). La priorité absolue est freeze → observabilité → réseau vivant → chaos → endurance.

## Les rôles dans le réseau ToM (complet)

Chaque appareil qui rejoint le réseau peut recevoir un ou plusieurs de ces rôles. Personne ne choisit son rôle — le réseau décide.

| # | Rôle        | En clair                                                   | Comment on le reçoit                                                      | Comment on le perd                                                      |
|---|-------------|------------------------------------------------------------|---------------------------------------------------------------------------|-------------------------------------------------------------------------|
| 1 | Participant | Nœud de base. Envoie et reçoit des messages.               | Par défaut en arrivant                                                    | Promu relayeur si on contribue assez                                    |
| 2 | Relayeur    | Fait passer les messages des autres. Le facteur du réseau. | Le réseau nous promeut quand notre score de contribution dépasse un seuil | Rétrogradé si on arrête de contribuer (le score baisse de 5% par heure) |

| # | Rôle                         | En clair                                                                                                                      | Comment on le reçoit                         | Comment on le perd                                    |
|---|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------|-------------------------------------------------------|
| 3 | <b>Gardien de messages</b>   | Stocke temporairement les messages pour quelqu'un qui est hors ligne. Le message s'auto-détruit après livraison ou après 24h. | Automatique quand un destinataire est absent | Le message est livré ou expire                        |
| 4 | <b>Responsable de groupe</b> | Gère un groupe de discussion : distribue les messages à tous les membres, gère les entrées/sorties.                           | Le créateur du groupe, ou élu par le réseau  | Remplacé automatiquement s'il disparaît (~6 secondes) |
| 5 | <b>Remplaçant de groupe</b>  | Copie l'état du responsable. Prêt à prendre le relais immédiatement.                                                          | Désigné par le responsable                   | Promu responsable si le titulaire tombe               |
| 6 | <b>Suivant de groupe</b>     | Troisième dans la chaîne de secours. Se promeut remplaçant si celui-ci tombe aussi.                                           | Désigné par le remplaçant                    | Monte d'un cran si nécessaire                         |
| 7 | <b>Admin de groupe</b>       | Celui qui a créé le groupe. Peut inviter, exclure, dissoudre.                                                                 | Fixé à la création                           | Ne change jamais                                      |

## La pompe d'amorçage — c'est une fonction, pas un rôle

La pompe d'amorçage n'est pas un serveur spécial ni un rôle à créer. **C'est une fonction que tout nœud porte déjà :**

- Le relayer NAS fait du PeerPresent (il présente les nouveaux arrivants aux nœuds existants)
- La découverte locale (mDNS) permet aux nœuds sur le même réseau de se trouver
- Le DHT permet de trouver des nœuds via Internet

Quand le réseau est petit, la pompe d'amorçage est critique : si le seul nœud qui connaît les adresses tombe, les nouveaux ne peuvent plus entrer. Quand le réseau grandit, cette connaissance

se copie sur tous les nœuds — la pompe devient distribuée, donc plus personne n'est indispensable.

**Ce qu'on doit voir dans les logs** : quelle source d'amorçage a permis à chaque nœud d'entrer (découverte locale, PeerPresent via relayeur, DHT). Si le porteur initial tombe, les nouveaux trouvent-ils encore le réseau ?

## La conscience du réseau — c'est des métriques, pas un rôle

Chaque nœud sait déjà combien de voisins il a. On ne crée pas un rôle "vigie" — on **expose dans les logs** ce que chaque nœud sait :

- **Combien de nœuds actifs** il voit
- **Sa phase** : amorçage (peu de voisins) / croissance (voisins qui augmentent) / stable / déclin (voisins qui partent)
- **Ses rôles actifs** : participant, relayeur, gardien de messages, responsable/remplaçant de groupe
- **La distribution des rôles qu'il perçoit** : combien de relayeurs dans le réseau, combien de gardiens

C'est de l'observation dérivée de ce qui existe. Pas une nouvelle brique de protocole.

## Le principe des logs — non négociable

**Chaque nœud, quel que soit l'appareil, envoie ses logs en JSON structuré au MacBook Pro par UDP. Toujours. Par défaut. Sans configuration.**

Le MacBook Pro est la **tour de contrôle dev** — pas une dépendance produit codée en dur. En production, chaque nœud est autonome. Mais pour le développement et les tests, on a besoin de tout voir depuis un seul écran.

Format de chaque ligne de log :

```
{  
  "ts": "2026-04-01T14:30:00Z",  
  "node": "appletv",  
  "appareil": "tvos",  
  "event": "pair_trouvé",  
  "detail": "macbook-1",  
  "phase": "amorçage",  
  "taille_reseau": 3,  
  "role": "participant",  
  "source_amorcage": "peer_present"  
}
```

Tous les appareils utilisent **le même runtime, les mêmes primitives**. Pas de mode spécial campagne. Juste des labels de nœuds et des logs en plus.

## Plan d'exécution — 8 étapes

**Règle d'or : on ne passe pas à l'étape suivante si le critère de sortie de l'étape en cours n'est pas validé. Pas de raccourci. Pas de "on verra plus tard".**

### ÉTAPE 1 — Sauver le travail

**But :** Commiter et pousser tout ce qui traîne.

**Faire :**

1. Ajouter les 8 fichiers modifiés
2. Commiter : "sauvegarde complète — amorçage, tvOS, apprentissage de routes, docs campagne"
3. Pousser sur la branche
4. Vérifier compilation et tests

**C'est bon quand :** le dépôt est propre, le push est fait, les tests passent.

**Si ça rate :** corriger ce qui bloque avant de pousser. On ne saute pas cette étape.

## ÉTAPE 2 — Donner des yeux au réseau

**But :** Voir TOUT ce qui se passe, en temps réel, depuis le MacBook Pro. Plus jamais "je ne vois pas les logs".

**Faire :**

### A. Normaliser un format de log JSON commun Rust / Swift

Un seul format. Mêmes champs partout. Que ce soit un iPhone, un NAS, ou un MacBook : même structure.

Champs obligatoires dans chaque ligne :

- horodatage, nom du nœud, type d'appareil
- événement, détail
- phase du réseau perçue par ce nœud
- taille du réseau vue par ce nœud
- rôle actuel du nœud
- source d'amorçage (si événement de découverte)

### B. Nœuds Rust : logs JSON par défaut en mode bot

Modifier `crates/tom-tui/src/main.rs` :

- Les logs sortent en JSON (pas de texte brut) en mode bot
- Chaque nœud porte un nom lisible passé en paramètre
- Envoi UDP au collecteur central en plus du fichier local

### C. Appareils Apple : mêmes logs JSON via UDP

Modifier `apps/tom-node-tvos/TomNode/Services/TomNodeService.swift` :

- Les logs UDP passent au format JSON structuré avec les mêmes champs que Rust

### D. Collecteur central sur MacBook Pro

Créer `scripts/collect-logs.sh` :

- Écoute UDP
- Écrit les logs dans des fichiers par nœud + un fichier fusionné
- Lisible directement dans VSCode par Claude ou GPT 5.4

## E. Page d'état locale par nœud

Chaque nœud expose une petite page web locale (un port HTTP) qui montre : identité, phase, voisins, rôles, groupes, métriques.

Fichiers : `crates/tom-protocol/src/runtime/metrics.rs` (enrichir les métriques),  
`crates/tom-tui/src/main.rs` (exposer le serveur)

### C'est bon quand :

- On lance un nœud, on voit ses logs JSON dans VSCode en moins de 5 secondes
- On lance 3 nœuds, le collecteur montre les 3 flux entrelacés
- Chaque ligne de log contient : phase du réseau, taille du réseau, rôle du nœud
- La page d'état locale répond et montre l'état du nœud

**Si ça rate** : on ne passe pas à l'étape suivante. Sans les yeux, on est aveugle.

## ÉTAPE 3 — Faire naître un noyau stable

**But** : Le réseau naît grâce au module d'amorçage embarqué dans chaque nœud. Pas de configuration manuelle. Le module d'amorçage fait son travail, ensuite le réseau vit sa vie.

### Noyau permanent (tourne en continu) :

- NAS : 1 relayeur réseau + 1 nœud ToM complet
- MacBook Pro : 3 nœuds ToM complets ("mbp-1", "mbp-2", "mbp-3")
- MacBook Air : 1 nœud ToM complet ("macair")

### → 5 nœuds minimum dès le départ

Séquence : NAS démarre → MacBook Pro lance ses 3 nœuds → MacBook Air lance le sien → on regarde les logs → les nœuds se trouvent tout seuls → les rôles commencent à se distribuer.

### C'est bon quand :

- 5 nœuds visibles dans le collecteur central
- Les nœuds se sont trouvés via l'amorçage automatique (visible dans les logs : source de découverte)
- Au moins 1 nœud promu relayeur
- Convergence en moins de 2 minutes

### **Si ça rate :**

- Les nœuds ne se trouvent pas → regarder les logs : la découverte locale fonctionne-t-elle ? Le relayeur NAS répond-il ?
- **On ne continue pas tant que 5 nœuds ne communiquent pas.**

## **ÉTAPE 4 — Faire entrer les satellites**

**But :** Les appareils mobiles (Apple TV, iPhone, iPad) rejoignent le réseau vivant quasi automatiquement.

### **Appareils à brancher :**

- Apple TV : 1 nœud (toujours allumé, traité comme participant normal)
- iPhone : 1 nœud (Wi-Fi ou 4G/5G, rejoint et quitte librement)
- iPad : 1 nœud (Wi-Fi, rejoint et quitte librement)

→ **Jusqu'à 8 nœuds au total**

Les satellites rejoignent le réseau déjà vivant. Ils ne font que se brancher — le module d'amorçage leur fait trouver les nœuds existants.

### **C'est bon quand :**

- Chaque appareil rejoint en moins de 2 minutes sans toucher de configuration
- Les logs montrent la taille du réseau qui augmente à chaque entrée
- Les logs montrent quelle source d'amorçage a permis l'entrée (PeerPresent, découverte locale, DHT)

### **Si ça rate :**

- Un appareil ne rejoint pas → vérifier que le relayeur NAS est joignable depuis son réseau
- **On ne continue pas tant que les satellites ne rejoignent pas.**

## **ÉTAPE 5 — Prouver le handoff d'amorçage**

**But :** Prouver que le réseau n'a plus besoin du porteur initial de l'amorçage. La connaissance du réseau s'est copiée sur plusieurs nœuds.

### **Faire :**

1. Le réseau tourne avec 5+ nœuds
2. Couper le NAS (le premier porteur d'amorçage)
3. Lancer un nouveau nœud (ex: relancer un des nœuds MacBook Pro)
4. Observer : le nouveau nœud trouve-t-il le réseau SANS le NAS ?
5. Remettre le NAS en route

### **C'est bon quand :**

- Le réseau continue après la coupure du NAS
- Un nouveau nœud trouve le réseau via les nœuds survivants
- Les logs montrent que la source d'amorçage n'est plus le NAS mais un autre nœud
- Le NAS revient et réintègre le réseau normalement

### **Si ça rate :**

- Le nouveau nœud ne trouve personne → l'amorçage dépend encore d'un point unique. Il faut vérifier que les nœuds survivants exposent bien leurs adresses aux nouveaux arrivants.
- **Si le handoff ne fonctionne pas, c'est un vrai trou dans le protocole. On corrige avant de continuer.**

## **ÉTAPE 6 — Observer la distribution des rôles**

**But :** Vérifier que les rôles se distribuent correctement et que les groupes fonctionnent avec leur chaîne de secours complète.

### **Faire :**

#### **A. Laisser tourner le réseau avec du trafic automatique**

Les nœuds en mode bot échangent des messages. Les scores montent. Les promotions arrivent.

#### **B. Création automatique d'un groupe**

Quand un nœud détecte 3+ voisins, il crée un groupe et invite tout le monde. Pas de commande manuelle.

#### **C. Observer dans les logs :**

- Combien de relayeurs promus



- Qui est responsable du groupe, qui est remplaçant, qui est suivant
- Est-ce que la pompe d'amorçage est encore nécessaire ou si le réseau se maintient seul
- Les messages de groupe arrivent à tous les membres

### **C'est bon quand :**

- Au moins 2 nœuds promus relayeurs
- Un groupe existe avec 5+ membres
- La chaîne de secours du groupe est complète (responsable + remplaçant + suivant)
- Les logs montrent une taille stable et des rôles distribués

### **Si ça rate :**

- Pas de promotion → le seuil est trop haut pour un petit réseau. On le baisse.
- Pas de chaîne de secours → il faut d'abord des relayeurs promus.
- **On ne continue pas tant que les rôles ne se distribuent pas.**

## **ÉTAPE 7 — Faire du chaos**

**But :** Casser le réseau volontairement et vérifier qu'il se répare tout seul.

### **A. Couper le responsable du groupe**

- Le remplaçant prend le relais en quelques secondes
- Les messages de groupe reprennent

### **B. Couper le relayeur NAS**

- Les nœuds locaux se retrouvent par découverte locale
- On relance le relayeur : tout le monde se reconnecte

### **C. Sortir et revenir avec l'iPhone**

- Couper l'app → le réseau continue sans lui
- Relancer → l'iPhone rejoint, reçoit un rôle
- Vérifier que le rôle attribué tient compte de la qualité du réseau (vitesse, latence)

### **D. Couper le MacBook Pro (3 nœuds d'un coup)**

- Les rôles se redistribuent entre les survivants
- Les logs montrent la phase du réseau qui change

### **C'est bon quand :**

- Remplacement du responsable de groupe en moins de 10 secondes
- Messages reprennent en moins de 15 secondes
- Reconnexion après coupure relayeur en moins de 1 minute
- L'iPhone rejoint et quitte librement sans rien casser
- Après la perte du MacBook Pro : le réseau continue avec les survivants

### **Si ça rate :**

- Le remplacement ne se fait pas → la chaîne de secours n'existait pas (retour à l'étape 6)
- Les nœuds ne se reconnectent pas → problème de reconnexion automatique, à corriger
- **Si le réseau ne survit pas aux pannes, c'est un trou dans le protocole. On diagnostique et on corrige.**

## **ÉTAPE 8 — Endurance**

**But :** Le réseau tient dans la durée.

### **Faire :**

- Laisser le noyau permanent actif (NAS + MacBook Pro + MacBook Air)
- Rejoindre et quitter avec l'iPhone et l'iPad plusieurs fois dans la journée
- Observer les logs sur la durée

### **C'est bon quand :**

- Le réseau tourne 1h sans intervention
- Puis 6h
- Les rôles restent stables quand personne ne bouge
- Les entrées/sorties mobiles n'accumulent pas d'erreurs
- La taille du réseau dans les logs correspond à la réalité

### **Si ça rate :**

- Fuite mémoire → analyser quel nœud grossit
- Erreurs qui s'accumulent → lire les logs, identifier le pattern
- **C'est la dernière étape. Si elle tient, on a un réseau vivant.**

# Ce qu'il faut créer ou modifier

| Étape | Fichier                                                               | Ce qu'on fait                                                         |
|-------|-----------------------------------------------------------------------|-----------------------------------------------------------------------|
| 1     | git                                                                   | Commit + push                                                         |
| 2     | <code>crates/tom-tui/src/main.rs</code>                               | Logs JSON par défaut en mode bot, nom du nœud, envoi UDP, page d'état |
| 2     | <code>apps/tom-node-tvos/TomNode/Services/TomNodeService.swift</code> | Logs UDP en JSON structuré (même format que Rust)                     |
| 2     | <code>scripts/collect-logs.sh</code> (nouveau)                        | Collecteur central UDP → fichiers                                     |
| 2     | <code>crates/tom-protocol/src/runtime/metrics.rs</code>               | Ajouter phase réseau, taille réseau, rôles dans les métriques         |
| 4     | <code>crates/tom-tui/src/main.rs</code>                               | Création automatique de groupe quand 3+ voisins                       |

## Verrouillages importants (rappel GPT 5.4)

1. **Pompe d'amorçage** = fonction distribuée que tout nœud porte déjà. Pas un serveur spécial.
2. **Conscience du réseau** = métriques dans les logs. Pas un nouveau rôle protocolaire.
3. **MacBook Pro collecteur** = tour de contrôle dev. Pas une dépendance produit codée en dur.
4. **Pas de mode campagne bricolé** : même runtime, mêmes primitives, juste plus d'observabilité et de labels.

# Ce qu'on vérifie à la fin

1. Le travail est sauvegardé et poussé
2. On voit les logs de TOUS les nœuds dans VSCode en temps réel
3. 5+ nœuds communiquent
4. Les satellites (iPhone, iPad, Apple TV) rejoignent et quittent librement
5. Le porteur initial d'amorçage peut tomber — le réseau continue
6. Les rôles se distribuent automatiquement
7. On coupe un nœud important → le réseau se répare
8. Le réseau tourne 1h+ sans intervention